

Project Report: Smart E-Commerce System

Vo Khanh Tran - 10423177
Doan Thi Minh Ngoc - 104240572
Duong Huynh Anh Minh - 104240347

June 18, 2026

1. Introduction

1.1. Project Overview

This project presents a console-based Smart E-Commerce System developed in C++ leveraging Object-Oriented Programming (OOP) principles. The system simulates a specialized online retail environment—focusing on an Apple accessories store (featuring iPhone cases, iPad cases, AirPods accessories, chargers, and screen protectors). It provides a dual-interface experience: a Customer side for browsing, managing a shopping cart, checking out, and receiving personalized recommendations; and an Admin side for catalog management and extracting business insights (such as most viewed products, best-selling items, active users, and specific recommendation score breakdowns).

```
=====
Main Menu
=====
1. Login
2. Register
0. Exit
Enter your choice: |
```

Figure 1. Main Application Menu

1.2. Core Objectives

- Develop a functional console-based e-commerce simulation.
- Apply robust C++ OOP concepts including encapsulation, inheritance, and composition.
- Manage core business entities: products, customers, administrators, shopping carts, and orders.
- Implement a lightweight, dependency-free data persistence layer using TXT files.
- Track and analyze customer behavior via interaction histories (views and purchases).
- Engineer a scoring-based, rule-driven product recommendation engine using Jaccard similarity.
- Provide comprehensive administrative tools for inventory control and business statistics.

2. System Architecture and OOP Design

2.1. System Actors

- **Customer:** The primary end-user. Customers can authenticate, register, browse the catalog, perform token-based searches, search by price range, view best-selling products, manage their ShoppingCart (update quantity, remove items, clear cart), process checkouts, view order histories, and receive tailored product recommendations based on their interactions.
- **Admin:** The system manager. Administrators possess privileges to execute CRUD operations on the product inventory, view color-coded stock statuses, and access advanced business insights. The admin inventory flow utilizes specific command inputs: A to add a product, ID to edit a product, and D + ID to delete a product.

2.2. Architectural Pattern

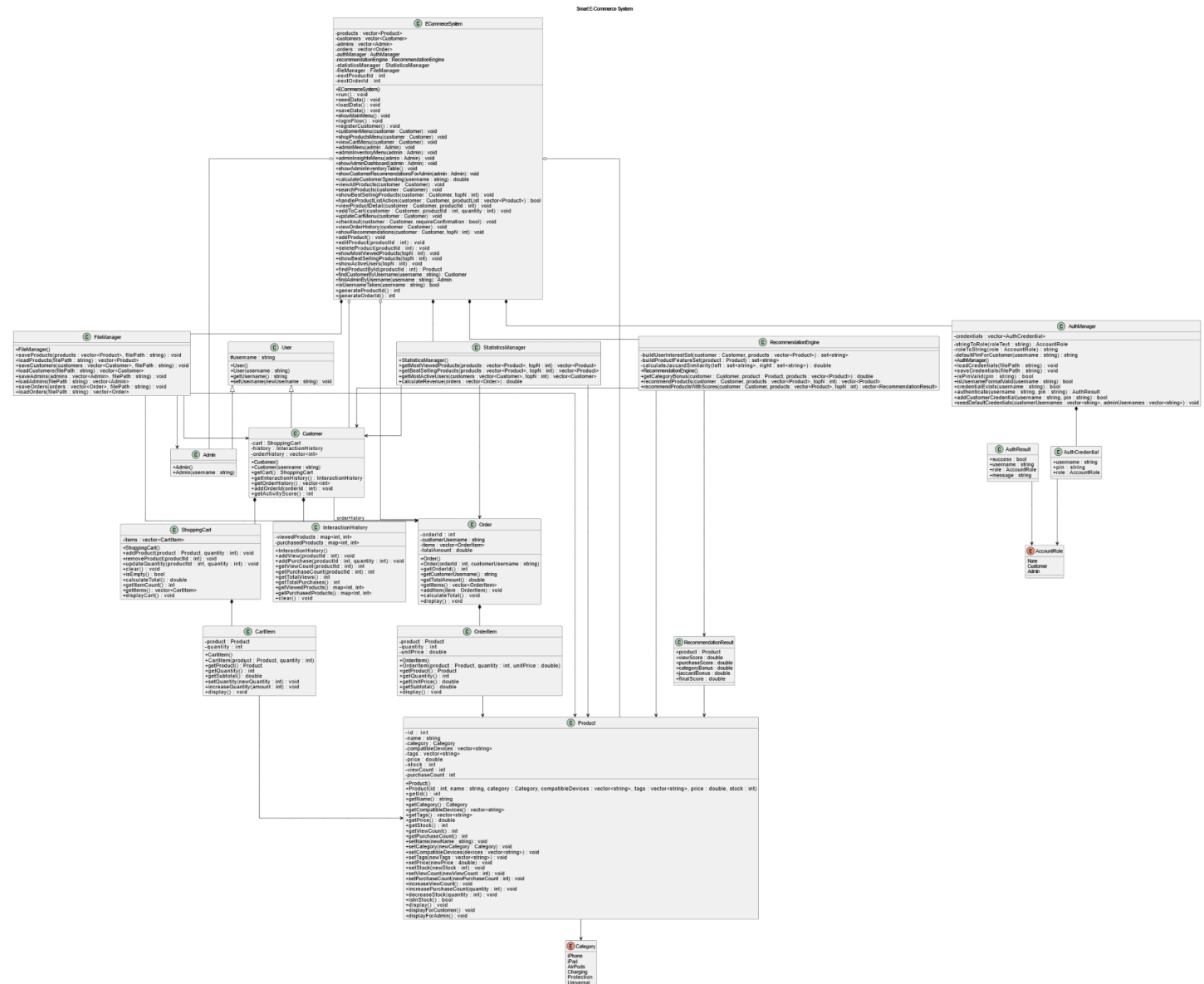


Figure 2. UML Class Diagram of the Smart E-Commerce System

The application utilizes a Controller/Manager architectural pattern. The `ECommerceSystem` class acts as the central coordinator (Dispatcher). It controls the application flow, manages user menus, and holds the primary data collections (vectors of products, customers, admins, and orders). It delegates specific, complex tasks to specialized manager modules:

- **AuthManager:** Supports the unified login flow by validating username/PIN credentials and returning the authenticated account role.
- **FileManager:** Manages TXT-based data persistence.
- **RecommendationEngine:** Computes and ranks personalized product suggestions.
- **StatisticsManager:** Aggregates business data for administrative insights.

2.3. Applied OOP Principles

- **Encapsulation:** Class attributes (e.g., Product stock, views, and price) are strictly private. State changes are controlled via public mutator methods like `decreaseStock()` and `increaseViewCount()`, safeguarding data integrity.

```
void Product::decreaseStock(int quantity) {  
    if (quantity > 0 && quantity <= stock) {  
        stock -= quantity;  
    }  
}
```

Figure 3. Encapsulation Example in Product::decreaseStock()

- **Inheritance:** The system implements a base User class containing shared attributes like the username. Both Customer and Admin inherit from User, adhering to the DRY principle.
- **Composition & Aggregation:** A Customer owns a ShoppingCart and an InteractionHistory. A ShoppingCart is composed of multiple CartItem objects, and an Order is composed of multiple OrderItem objects.

3. Technical Implementation and Module Breakdown

3.1. Authentication and Registration (AuthManager)

The AuthManager supports the unified login flow by validating credentials and returning the authenticated account role. `ECommerceSystem::loginFlow()` receives console input and routes users to the correct menu, while AuthManager stores credentials using the `AuthCredential` structure (username, PIN, and `AccountRole` enum: None, Customer, Admin). The registration flow is available for Customer accounts only. Usernames must start with a lowercase letter and may contain only lowercase letters or digits. PINs must be stored as strings and must contain exactly 4 digits, which prevents issues with PINs that start with 0. During login, AuthManager authenticates the username and PIN, then returns the account role so `ECommerceSystem` can open either the Customer menu or the Admin menu.

3.2. Catalog and Inventory Management

- **Product:** Represents individual items. Beyond basic data, it tracks `viewCount` and `purchaseCount`.
- **Category:** Implemented as a strongly-typed enum.
- **Admin Inventory Flow:** The admin can view the inventory table, add, edit, and delete products. Deletion triggers a danger warning requiring confirmation.

```
[Admin: admin1]
```

```
Admin Portal > Inventory Management
```

```
=====
```

Inventory Management

```
=====
```

ID	Product Name	Category	Price	Sold	Stock	Views
1	iPhone 15 Clear Case	iPhone	250000	18	10	3
2	iPhone 15 Pro Silicone Case	iPhone	320000	0	15	2
3	iPad Air Folio Case	iPad	450000	3	7	0
4	iPad Paperlike Screen Protector	Protection	290000	1	18	3
5	Apple Pencil Tips Pack	iPad	190000	3	Sold out	1
7	20W USB-C Charger	Charging	490000	6	11	3
8	USB-C to USB-C Cable 1m	Charging	180000	1	29	0
9	MagSafe Power Bank	Charging	890000	2	8	1
10	Tempered Glass for iPhone 15	Protection	150000	0	22	0
11	Tempered Glass for iPad Air	Protection	240000	0	16	0
12	Laptop Sleeve 13 inch	Universal	390000	1	8	1
13	iPad Pro Clear Case	iPad	280000	3	1	2
14	iPhone 17 Pro Max Silicone Case	iPhone	200000	0	18	0

```

A      - Add product
ID     - Edit product, example: 5
D + ID - Delete product, example: D5
0      - Back

Enter your choice:

```

Figure 4. Admin Inventory Management Table with Color-Coded Stock

3.3. Shopping Cart and Order Processing

Customers update their carts using dynamically generated item numbers rather than complex product IDs. This improves usability because customers do not need to remember product IDs after adding items to the cart. The cart supports setting a new quantity, removing a selected item, and clearing the entire cart with confirmation. Quantity validation is performed against the current product stock, and invalid quantities are rejected before the cart is updated. The checkout() function executes a critical stock validation just before finalization to account for potential backend inventory changes. If the product no longer exists or the stock is insufficient, checkout fails and the system clearly displays the product name, quantity in cart, and available stock. After a successful checkout, the system decreases product stock, increases the product sold count, records the purchase in the customer's InteractionHistory, creates an Order with OrderItem entries, adds the order ID to the customer's order history, clears the cart, and saves the updated data to TXT files.

```

[User: user1 | Cart: 3 items]
=====
Home > Customer Main Menu > View Cart
=====
Your Cart
=====
1. USB-C to USB-C Cable 1m | Qty: 2 | Price: 180000 | Subtotal: 360000
2. iPhone 17 Pro Max Silicone Case | Qty: 1 | Price: 200000 | Subtotal: 200000
Total: 560000
  1. Checkout
  2. Update cart
  0. Back

  Enter your choice: 1
Proceed to checkout? (Y/N): Y
=====
Order Summary
=====
Order ID: 11
Customer Username: user1
USB-C to USB-C Cable 1m x 2 @ 180000 = 360000
iPhone 17 Pro Max Silicone Case x 1 @ 200000 = 200000
Total Amount: 560000
=====
Checkout successful.

Press Enter to continue...

```

Figure 5. Successful Checkout and Order Summary

3.4. Interaction Tracking (InteractionHistory)

This module silently tracks user behavior utilizing two map structures:

- **viewedProducts:** Stores productId -> view count.
- **purchasedProducts:** Stores productId -> purchased quantity.

3.5. Data Persistence (FileManager)

The system intentionally avoids heavy database dependencies in favor of lightweight TXT files. The project does not use JSON or a database; all persistence is handled through plain TXT files. This design keeps the project simple, dependency-free, and easy to inspect manually, which is suitable for a console-based C++ coursework project. The tradeoff is that TXT files do not provide advanced database features such as transactions, concurrent access control, indexing, or automatic schema validation. The data is divided among the following files:

- **products.txt:** Product data, including product ID, product name, category, compatible devices, tags, price, stock, global view count, and sold count.
- **customers.txt:** Customer username, viewed products, purchased products, and order history. Viewed and purchased products are stored using productId-to-count mappings.
- **admins.txt:** Admin usernames.
- **users.txt:** Authentication credentials, including username, 4-digit PIN, and account role.

- **orders.txt:** Completed orders, including order ID, customer username, total amount, and purchased item snapshots.

3.6. Detailed Code Structure and Architectural Highlights

The Smart E-Commerce System is implemented using a modular Object-Oriented Programming (OOP) approach in C++. The architecture follows a Controller-Manager style, where ECommerceSystem acts as the main controller and delegates specialized responsibilities to manager classes. This design improves separation of concerns by moving authentication, file persistence, recommendation, and statistics into dedicated modules.

3.6.1. Centralized Dispatcher: ECommerceSystem

The ECommerceSystem class functions as the central coordinator of the system. It owns the primary in-memory data structures, including vectors of Product, Customer, Admin, and Order objects. It also manages the main application flow, user menus, customer actions, admin actions, checkout processing, and routing after login.

ECommerceSystem delegates specialized tasks to AuthManager, RecommendationEngine, StatisticsManager, and FileManager. Most high-level coordination happens through ECommerceSystem rather than direct interaction between manager classes. The program entry point, main.cpp, simply creates an ECommerceSystem object and calls run(). The run() method then loads data, seeds default data when needed, saves initial state, and starts the main menu loop.

3.6.2. Role-Based Access Control and Authentication

The authentication mechanism is supported by AuthManager and coordinated by ECommerceSystem::loginFlow(). AuthManager uses the AccountRole enum class to distinguish between Customer and Admin accounts. After successful credential validation, ECommerceSystem routes the user to either the Customer menu or the Admin menu based on the returned role.

To maintain valid registration data, the system enforces strict validation rules:

- **Username Constraints:** The username must begin with a lowercase letter and may contain only lowercase letters and digits.
- **PIN Validation:** PINs are stored as strings rather than integers to preserve leading zeroes, such as "0123". Each PIN must contain exactly 4 digits.

3.6.3. Dynamic Cart Mapping and Concurrency-Safe Checkout

A prominent usability feature within the ShoppingCart module is the dynamic item mapping. Rather than forcing users to manipulate complex system-generated productIds, the cart dynamically assigns sequential item numbers during the active session.

Crucially, the checkout process mitigates pseudo-concurrency issues. Since an administrator might modify inventory levels while a customer is still browsing, the checkout() function executes a critical pre-flight validation check against the central Product vector immediately before finalizing the transaction.

```
for (const auto& item : customer.getCart().getItems()) {  
    Product* product = findProductById(item.getProduct().getId());
```

```

    if (product == nullptr) {
        cout << "Checkout failed.\n\n"
             << "Product: " << item.getProduct().getName() << "\n"
             << "This product is no longer available.\n"
             << "Please update your cart before checkout.\n";
        updateCartMenu(customer);
        return;
    }

    if (item.getQuantity() > product->getStock()) {
        cout << "Checkout failed.\n\n"
             << "Product: " << product->getName() << "\n"
             << "Quantity in cart: " << item.getQuantity() << "\n"
             << "Available stock: " << product->getStock() << "\n"
             << "Please update your cart before checkout.\n";
        updateCartMenu(customer);
        return;
    }
}

```

Figure 6. Final Stock Validation Before Checkout

If the validation succeeds, the system atomically decreases stock, increments the product's purchaseCount, updates the user's InteractionHistory, generates an Order record, and flushes the state to persistent storage.

3.6.4. Rule-Based Personalization: The Recommendation Engine

Instead of relying on computationally heavy machine learning libraries, the system implements a highly lightweight and deterministic recommendation algorithm via the RecommendationEngine class. It builds a "User Interest Set" dynamically from the metadata (categories, compatible devices, tags) of products the user has previously interacted with.

The algorithm evaluates all in-stock products by computing a composite score based on the following mathematical model:

$$FinalScore = S_{view} + S_{purchase} + B_{category} + B_{jaccard}$$

Where:

- S_{view} (View Score): Historical view count multiplied by a 0.2 weight.
- $S_{purchase}$ (Purchase Score): Historical purchase quantity multiplied by a 0.8 weight.
- $B_{category}$ (Category Bonus): A flat +1.0 addition if the item matches the user's preferred category, calculated from both viewed products and purchased products, with purchases weighted more strongly.

- $B_{jaccard}$ (*Jaccard Bonus*): Evaluates the intersection over union of the User Interest Set (A) and the Product Feature Set (B):

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

This transparent scoring mechanism is also exposed to the administration side. Using the `showCustomerRecommendationsForAdmin()` function, administrators can audit the exact mathematical breakdown of any user's recommendation pipeline.

```
result.product = product;

    result.viewScore = history.getViewCount(product.getId()) * 0.2;

    result.purchaseScore = history.getPurchaseCount(product.getId()) * 0.8;

    result.categoryBonus = getCategoryBonus(customer, product, products);

    result.jaccardBonus =
                                calculateJaccardSimilarity(userInterestSet,
buildProductFeatureSet(product));

    result.finalScore = result.viewScore
        + result.purchaseScore
        + result.categoryBonus
        + result.jaccardBonus;
```

Figure 7. Recommendation Score Calculation

3.6.5. Behavioral Tracking and Business Intelligence

The InteractionHistory module acts as a silent observer, capturing granular user behavior through internal map structures (`productId` $\rightarrow$ `viewCount` / `purchaseCount`). Every invocation of `viewProductDetail()` synchronously increments both the global product view counter and the localized user history.

Leveraging this data, the StatisticsManager aggregates business insights, allowing administrators to query top-selling and most-viewed items. Furthermore, it quantifies user engagement through a defined `activityScore`:

$$ActivityScore = V_{total} + (P_{total} * 4)$$

Where V_{total} represents total item views and P_{total} represents total units purchased. This allows administrators

to accurately rank and identify highly active customers.

```
int Customer::getActivityScore() const {  
  
    return history.getTotalViews() + (history.getTotalPurchases() * 4);  
  
}
```

Figure 8. Active User Score Calculation

3.6.6. Lightweight Persistence Layer

To adhere to the constraints of a purely console-based academic project, the FileManager circumvents heavy SQL databases or external JSON parsers in favor of a bespoke text-based persistence layer. Data serialization is distributed across distinct .txt files (e.g., products.txt, customers.txt, users.txt). While this approach sacrifices ACID compliance and transactional safety, it yields a zero-dependency, easily auditable codebase. The FileManager utilizes custom string tokenization functions to encode complex relationships, such as serializing `std::vector` and `std::map` objects into flat comma-separated strings for storage.

4. Recommendation Engine

The core of the system's personalization is the RecommendationEngine, which employs a rule-based, scoring algorithm rather than machine learning.

4.1. Scoring Formula

The engine calculates a finalScore for every in-stock product using the following components:

$$finalScore = viewScore + purchaseScore + categoryBonus + jaccardBonus$$

- **viewScore:** $view_{count} * 0.2$ (where $view_{count}$ is the current customer's view count for that product in InteractionHistory).
- **purchaseScore:** $purchase_{count} * 0.8$ (where $purchase_{count}$ is the quantity purchased by the current customer, not the overall number of buyers).
- **categoryBonus:** Adds 1.0 if the product aligns with the customer's historically preferred category.
- **jaccardBonus:** A similarity score comparing the user's interest set with the product's feature set. Only in-stock products are considered for recommendations.

4.2. Algorithmic Approach: Jaccard Similarity

The algorithm builds a "User Interest Set" dynamically from the categories, compatible devices, and tags of products the user has interacted with. It then builds a "Product Feature Set" for the item being evaluated.

The system calculates the intersection over the union of these two sets:

$$Jaccard = \frac{|Intersection|}{|Union|}$$

```

double RecommendationEngine::calculateJaccardSimilarity(
    const set<string>& left,
    const set<string>& right
) const {
    if (left.empty() && right.empty()) {
        return 0.0;
    }

    int intersectionCount = 0;
    for (const auto& value : left) {
        if (right.count(value) > 0) {
            ++intersectionCount;
        }
    }

    int unionCount = static_cast<int>(left.size() + right.size() -
intersectionCount);
    if (unionCount == 0) {
        return 0.0;
    }

    return static_cast<double>(intersectionCount) / unionCount;
}

```

Figure 9. Jaccard Similarity Calculation

5. Discussion and System Analysis

5.1. Administrative Business Insights

The StatisticsManager equips administrators with powerful analytical tools:

- Most Viewed Products & Best-Selling Products
- Active Users: Ranked by a calculated activity score, defined as:
$$activityScore = totalViews + totalPurchases * 4$$
- Recommendation Transparency: Using recommendProductsWithScores(), admins can input a specific username to view a detailed breakdown of the recommendation algorithm's output.

```
[Admin: admin1]
-----
Admin Portal > Business Insights > Customer Recommendations
-----
=====
Recommendation Scores
=====
Customer: user1
Formula: final = view_score + purchase_score + category_bonus + jaccard_bonus
view_score = view_count * 0.2 | purchase_score = purchase_count * 0.8
-----
```

Rank	ID	Product Name	View	Buy	Cat	Jaccard	Final
1	1	iPhone 15 Clear Case	0.20	14.40	1.00	0.19	15.79
2	13	iPad Pro Clear Case	0.40	2.40	0.00	0.19	2.99
3	8	USB-C to USB-C Cable 1m	0.20	2.40	0.00	0.26	2.86

Figure 10. Admin Recommendation Score Breakdown

5.2. System Limitations

- **User Interface:** Lacks a GUI.
- **Security:** PINs are currently stored as plain text.
- **Data Integrity:** TXT-based persistence lacks transactional safety.
- **Algorithmic Bounds:** The recommendation system is strictly rule-based.

6. Conclusion

The Smart E-Commerce System successfully delivers a functional console-based e-commerce simulation. It supports the main customer workflows and administrative duties. The codebase applies OOP principles and separates several responsibilities into manager classes, although ECommerceSystem remains the central and largest controller. The recommendation logic is explainable for administrators through score breakdowns, while customers only see the final recommended products, providing a practical mechanism for personalized product recommendations without the overhead of external ML libraries.

7. References

1. Jaccard Similarity: GeeksforGeeks, "Jaccard Similarity," available at: <https://www.geeksforgeeks.org/python/jaccard-similarity/> (Accessed: May 18, 2026).